Design Document

TerraMind AI - AI-powered agriculture crop advisory assistant

1. Problem statement

Agricultural professionals hold thousands of historical records (crop, season, state, soil, weather, inputs, yield) but cannot quickly find comparable cases or judge which crops have done well under similar conditions. TerraMind retrieves similar historical cases from natural-language descriptions and turns them into evidence-backed, explainable crop recommendations - decision support, never a yield guarantee.

2. Architecture

Offline: validate and merge the dataset, build case texts, embed with MiniLM, persist a FAISS index and a BM25 index, train the yield model. Online: a FastAPI service exposes an orchestrator that runs seven agents with real per-stage status; a React UI polls the job endpoint so progress shown is progress achieved. See architecture/Architecture_Diagram.pdf.

3. Technology choices and trade-offs

Choice	Why	Trade-off
FastAPI + Pydantic	Typed request/response validation, automatic OpenAPI	Sync pipeline run in worker threads instead of full async
SQLite (stdlib)	Zero-setup local persistence for queries, feedback, evaluations	Single-node; no migrations framework
FAISS flat inner product	55,000 vectors: exact search in milliseconds, no tuning	Would need IVF/HNSW beyond ~1M vectors
all-MiniLM-L6-v2	Small, CPU-friendly, adequate for short structured texts	Weak on numeric similarity (handled by structured candidates)
BM25 (rank-bm25)	Exact crop/state/season term matching	No stemming; vocabulary is small and controlled
scikit-learn HistGradientBoosting / RF	Strong tabular baselines, native NaN handling, quantile loss	No deep model; limited by state-level features
Claude (Anthropic SDK)	Structured extraction, grounded explanation, judging	Cost/latency; therefore used only where deterministic code cannot
React + Vite + Tailwind + Recharts	Fast dev loop, accessible components	Client-side polling instead of websockets

4. Dataset design

19,689 real records plus 35,311 labelled synthetic records (is_synthetic flag, 'TS-' ids) reach the 55,000 working size; models, statistics and suitability use real rows only (see Dataset_Details.pdf). Soil values are state constants and weather is state-year, joined and labelled as such. Stable case IDs are SHA-1 of the natural key, so a case keeps its identity across rebuilds; source_row links back to the raw CSV.

5. Case representation, chunking and embeddings

One CSV row is one case and one chunk: rows are ~60 words and splitting them would separate a crop from its conditions. The text is generated only from the record's own values and includes descriptive classes (e.g. 'acidic soil', 'high rainfall', 'high yield for this crop') so that lexical and semantic queries phrased in natural language can match. Embeddings are computed once at index-build time, L2-normalised, and stored with a manifest (dataset version, model, case ids). A stale index (dataset changed) is refused rather than silently mapped to wrong records.

6. Hybrid retrieval and ranking

• BM25 and semantic rankings are computed after applying structured metadata filters (crop, season, state, year and inclusive numeric ranges); filtered-out counts are returned.

• Fusion: Reciprocal Rank Fusion, score = sum 1/(60 + rank) over the two lists. It is a rank-based ordering score, not a calibrated probability.

• If one backend fails the other's results are returned with an explicit fallback notice; if both fail the API returns 503. No cases are ever fabricated.

• Because embeddings are weak on numbers, the retrieval agent adds structured candidates: the 2 cases per crop whose soil/weather values are closest to the user's.

• Prioritisation (src/rag/prioritizer.py): weighted sum of hybrid relevance 0.30, crop match 0.15, season 0.10, region 0.10, soil similarity 0.15, weather similarity 0.10, data completeness 0.10; components with no user input are 'not assessed' and weights are renormalised (unknown is never a match). Historical yield is deliberately NOT a component.

7. Agent collaboration

A central orchestrator sequences: validation, classification (Agent 1), retrieval + prioritisation (Agent 2), recommendation (Agent 3), then suitability (Agent 4), yield (Agent 5) and impact (Agent 6) in parallel threads with timeouts, then explanation and validation (Agent 7). Each stage records real status and duration; an optional agent that fails is reported and its siblings' results are kept. Agents exchange typed data (Pydantic / dicts), not free text. Claude is called for: query understanding (validated against dataset vocabulary, never used as a hard filter when inferred), the narrative explanation (given only a facts JSON) and the judge.

8. Prediction methodology

Target: yield, modelled as log1p(yield) because units span orders of magnitude between crops. Production is never a feature (leakage: yield ~ production/area). Two feature sets: 'core' (crop, season, state, year, soil, weather) and 'full' (+ log area, fertilizer/ha, pesticide/ha) used only when the user supplies area and inputs. Temporal split: train <= 2014 (14,406), validation 2015-2017 (3,104), test 2018-2020 (2,179); model chosen on validation log-RMSE, refit on train+validation, evaluated once on test. Prediction intervals come from quantile gradient boosting; empirical test coverage of the nominal 80% interval is 70% (full) / 73% (core), i.e. slightly under-covering - reported as is.

Feature set	Model	Test MAE	Test RMSE	Test R2	Test R2 (log1p)
core - baseline (crop median)	baseline	24.28	343.56	0.836	0.791
core - selected	hgb	17.04	290.32	0.883	0.957
full - baseline (crop median)	baseline	24.28	343.56	0.836	0.791
full - selected	hgb	10.05	151.23	0.968	0.956

Suitability risk: the dataset has no suitability labels, so a classifier would need invented labels. Instead a transparent rule-based 'historical envelope' compares each supplied condition with the 10th-90th percentile and min-max range in which the crop has been recorded (plus season/state presence and yield variability). The output is labelled as a heuristic, never a probability.

9. Guardrails

• Pydantic schemas with defensible hard ranges; unusual-but-possible values are flagged for confirmation rather than rejected; 64 KB body limit; query length limit.

• Prompt-injection: retrieved text and user queries are wrapped in blocks with a system rule that their content is data; suspicious phrasing is detected and surfaced as a warning; all LLM output is schema-validated; Claude is given no tools.

• The API key is read from .env by the backend only; it never appears in responses, logs or frontend variables.

• Failure handling: missing key, auth/billing/rate-limit/timeout errors, refusals and invalid JSON are typed (LLMError) with bounded retries and credential-free messages; the non-LLM pipeline keeps working.

10. Evaluation

Verified evaluation cases are built from the real dataset: relevance labels are deterministic filters (e.g. all records with the stated crop+state+season) and are re-verified on every run. Retrieval metrics (Precision@K, Recall@K, MRR, Hit@K) are computed only where labels exist. DeepEval faithfulness, answer relevancy, contextual precision and contextual recall run through a Claude adapter on pipeline answers; if the LLM is unavailable they are recorded as unavailable with the reason, never as a score. Yield-model metrics are held-out and reported separately. All runs persist to SQLite and artifacts/evaluation. Pass/fail rules: retrieval cases pass at R-precision >= 0.5 (Precision@10 is mathematically capped when fewer than 10 relevant cases exist; this rule was adopted after the first run exposed that cap - Precision@10 is still reported unchanged).

11. Feedback loop

Users submit helpful/not-helpful, rating, category and comment per recommendation; entries are stored with the query and supporting case ids. A summary service reports helpful rate, categories, trends and review candidates. The improvement workflow is deliberately manual: verify a failure, add it to data/evaluation/test_cases_extra.json, re-run evaluation, compare runs. There is no automatic retraining.

12. Limitations

• Only 19,689 real records (35k synthetic rows add retrieval volume but no new real evidence); soil is state-level; weather is state-year; yield units differ per crop.

• Recommendations reflect where crops have historically been recorded and how similar the conditions are - they are not agronomic prescriptions.

• Original-scale yield R2/RMSE are dominated by a few high-magnitude crops; log-scale metrics are reported alongside.

• Feature importance and correlations are associations; confounding by state and year is documented.

• LLM components depend on API availability and credit; without them the system degrades to deterministic parsing, template explanations and deterministic validation.